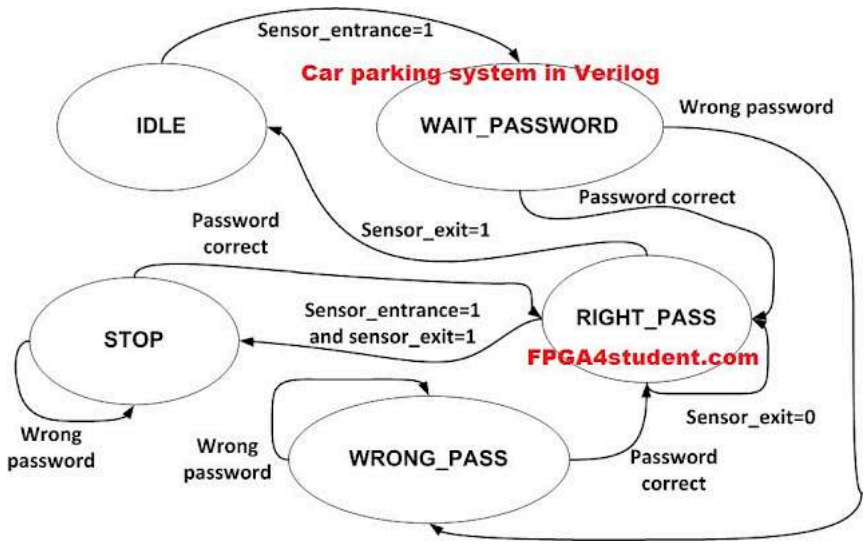
Verilog code for Car Parking System

This simple project is to implement a car parking system in Verilog. The Verilog code for the car parking system is fully presented.

In the entrance of the parking system, there is a sensor which is activated to detect a vehicle coming. Once the sensor is triggered, a password is requested to open the gate. If the entered password is correct, the gate would open to let the vehicle get in. Otherwise, the gate is still locked. If the current car is getting in the car park being detected by the exit sensor and another the car comes, the door will be locked and requires the coming car to enter passwords.



Verilog code for the car parking system:

```
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for car parking system
`timescale 1ns / 1ps
module parking_system(
                input clk,reset_n,
 input sensor_entrance, sensor_exit,
 input [1:0] password_1, password_2,
 output wire GREEN_LED,RED_LED,
 output reg [6:0] HEX_1, HEX_2
    );
 parameter IDLE = 3'b000, WAIT_PASSWORD = 3'b001, WRONG_PASS = 3'b010, RIGHT_PASS
 // Moore FSM : output just depends on the current state
 reg[2:0] current_state, next_state;
 reg[31:0] counter_wait;
 reg red_tmp,green_tmp;
 // Next state
 always @(posedge clk or negedge reset_n)
 begin
 if(~reset_n)
 current_state = IDLE;
 else
 current_state = next_state;
 end
 // counter_wait
 always @(posedge clk or negedge reset_n)
 begin
 if(~reset_n)
 counter_wait <= 0;
 else if(current_state==WAIT_PASSWORD)
 counter_wait <= counter_wait + 1;
```

```
 else
 counter_wait <= 0;
 end
 // change state
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
 always @(*)
 begin
 case(current_state)
 IDLE: begin
        if(sensor_entrance == 1)
 next_state = WAIT_PASSWORD;
 else
 next_state = IDLE;
 end
 WAIT_PASSWORD: begin
 if(counter_wait <= 3)
 next_state = WAIT_PASSWORD;
 else
 begin
 if((password_1==2'b01)&&(password_2==2'b10))
 next_state = RIGHT_PASS;
 else
 next_state = WRONG_PASS;
 end
 end
 WRONG_PASS: begin
 if((password_1==2'b01)&&(password_2==2'b10))
 next_state = RIGHT_PASS;
 else
 next_state = WRONG_PASS;
 end
 RIGHT_PASS: begin
 if(sensor_entrance==1 && sensor_exit == 1)
 next_state = STOP;
 else if(sensor_exit == 1)
 next_state = IDLE;
 else
 next_state = RIGHT_PASS;
 end
 STOP: begin
 if((password_1==2'b01)&&(password_2==2'b10))
 next_state = RIGHT_PASS;
 else
 next_state = STOP;
 end
 default: next_state = IDLE;
 endcase
 end
 // LEDs and output, change the period of blinking LEDs here
 always @(posedge clk) begin
 case(current_state)
 IDLE: begin
 green_tmp = 1'b0;
 red_tmp = 1'b0;
 HEX_1 = 7'b1111111; // off
 HEX_2 = 7'b1111111; // off
 end
 WAIT_PASSWORD: begin
 green_tmp = 1'b0;
 red_tmp = 1'b1;
 HEX_1 = 7'b000_0110; // E
 HEX_2 = 7'b010_1011; // n
 end
```

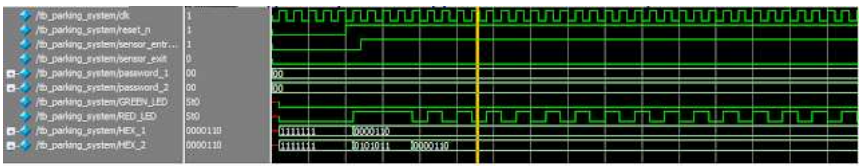
```
 WRONG_PASS: begin
 green_tmp = 1'b0;
 red_tmp = ~red_tmp;
 HEX_1 = 7'b000_0110; // E
 HEX_2 = 7'b000_0110; // E
 end
 RIGHT_PASS: begin
 green_tmp = ~green_tmp;
 red_tmp = 1'b0;
 HEX_1 = 7'b000_0010; // 6
 HEX_2 = 7'b100_0000; // 0
 end
 STOP: begin
 green_tmp = 1'b0;
 red_tmp = ~red_tmp;
 HEX_1 = 7'b001_0010; // 5
 HEX_2 = 7'b000_1100; // P
 end
 endcase
 end
 assign RED_LED = red_tmp  ;
 assign GREEN_LED = green_tmp;

endmodule
```

Testbench Verilog code for car parking system:

```
`timescale 1ns / 1ps
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for car parking system
module tb_parking_system;

  // Inputs
  reg clk;
  reg reset_n;
  reg sensor_entrance;
  reg sensor_exit;
  reg [1:0] password_1;
  reg [1:0] password_2;

  // Outputs
  wire GREEN_LED;
  wire RED_LED;
  wire [6:0] HEX_1;
  wire [6:0] HEX_2;
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
  // Instantiate the Unit Under Test (UUT)
  parking_system uut (
  .clk(clk),
  .reset_n(reset_n),
  .sensor_entrance(sensor_entrance),
  .sensor_exit(sensor_exit),
  .password_1(password_1),
  .password_2(password_2),
  .GREEN_LED(GREEN_LED),
  .RED_LED(RED_LED),
  .HEX_1(HEX_1),
 .HEX_2(HEX_2)
 );
 initial begin
 clk = 0;
 forever #10 clk = ~clk;
 end
```

```
 initial begin
 // Initialize Inputs
 reset_n = 0;
 sensor_entrance = 0;
 sensor_exit = 0;
 password_1 = 0;
 password_2 = 0;
 // Wait 100 ns for global reset to finish
 #100;
      reset_n = 1;
 #20;
 sensor_entrance = 1;
 #1000;
 sensor_entrance = 0;
 password_1 = 1;
 password_2 = 2;
 #2000;
 sensor_exit =1;

 // Add stimulus here
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
 end

endmodule
```

Simulation waveform for the car parking system in Verilog:



Car Parking System in VHDL
What is FPGA Programming? FPGA vs Software programming
Recommended and affordable Xilinx FPGA boards for students
Recommended and affordable Altera FPGA boards for students

Recommended Verilog projects:

1. **What is an FPGA? How Verilog works on FPGA**
2. Verilog code for FIFO memory
3. Verilog code for 16-bit single-cycle MIPS processor
4. Programmable Digital Delay Timer in Verilog HDL
5. Verilog code for basic logic components in digital circuits
6. Verilog code for 32-bit Unsigned Divider
7. Verilog code for Fixed-Point Matrix Multiplication
8. Plate License Recognition in Verilog HDL
9. Verilog code for Carry-Look-Ahead Multiplier
10. Verilog code for a Microcontroller
11. Verilog code for 4x4 Multiplier
12. Verilog code for Car Parking System
13. Image processing on FPGA using Verilog HDL
14. How to load a text file into FPGA using Verilog HDL
15. Verilog code for Traffic Light Controller
16. Verilog code for Alarm Clock on FPGA
17. Verilog code for comparator design
18. Verilog code for D Flip Flop
19. Verilog code for Full Adder
20. Verilog code for counter with testbench
21. Verilog code for 16-bit RISC Processor

22. Verilog code for button debouncing on FPGA
23. How to write Verilog Testbench for bidirectional/ inout ports
24. Tic Tac Toe Game in Verilog and LogiSim
25. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-1)
26. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-2)
27. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-3)
28. Verilog code for Decoder
29. Verilog code for Multiplexers
30. N-bit Adder Design in Verilog
31. Verilog vs VHDL: Explain by Examples
32. Verilog code for Clock divider on FPGA
33. **How to generate a clock enable signal in Verilog**
34. **Verilog code for PWM Generator**
35. **Verilog coding vs Software Programming**
36. **Verilog code for Moore FSM Sequence Detector**
37. **Verilog code for 7-segment display controller on Basys 3 FPGA**

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.